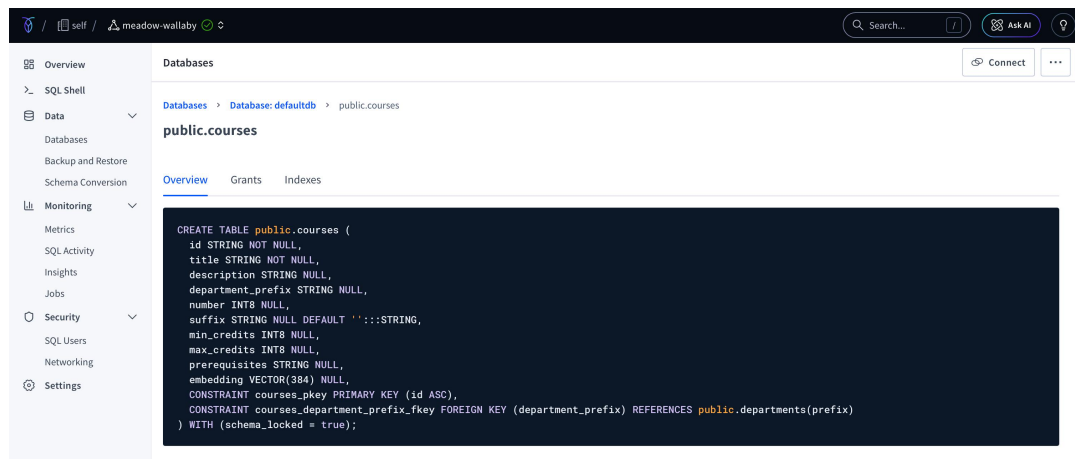


CockroachDB Agentic Memory Evidence

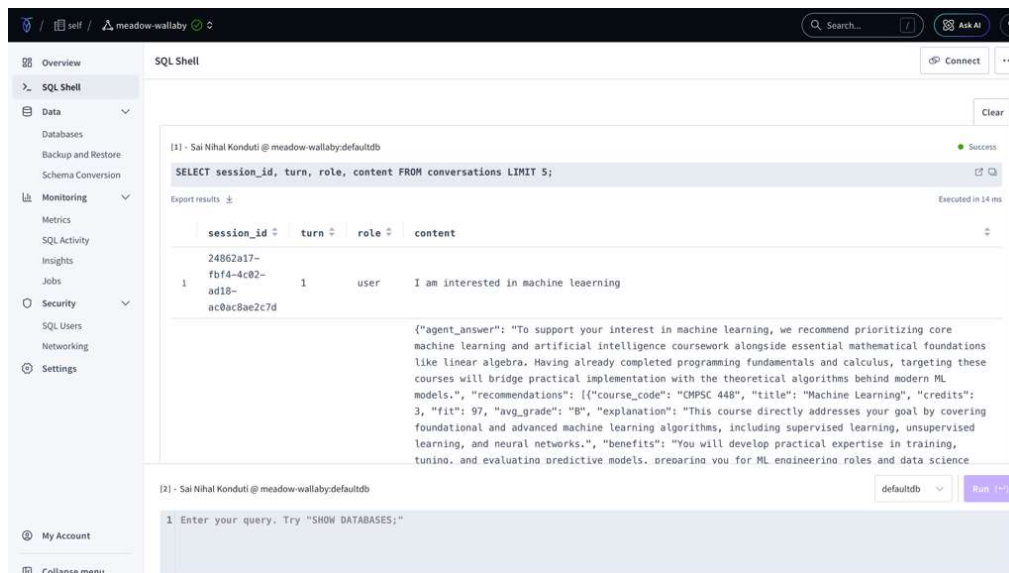
1. The pgvector Column (Proving Our Semantic Search)

- Where we went: In the CockroachDB console, we navigated to Databases -> clicked our database name -> clicked the courses table -> opened the Schema/Columns tab.
- What we captured: We took a screenshot showing the embedding column utilizing the VECTOR (or ARRAY) data type.
- Why we did this: We captured this to provide the ultimate proof that we used CockroachDB natively for AI semantic search, which we knew would be a massive selling point for the judges.



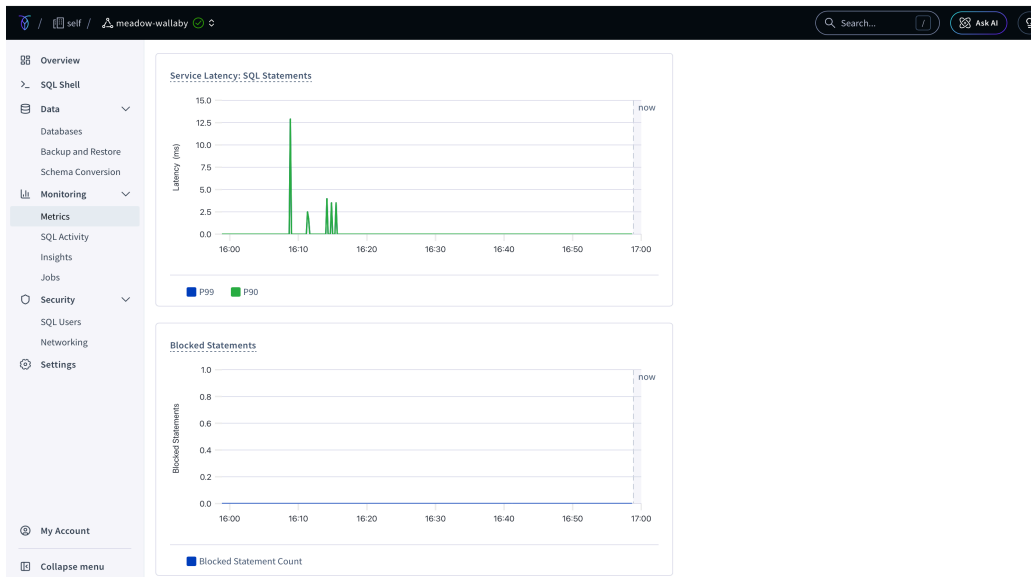
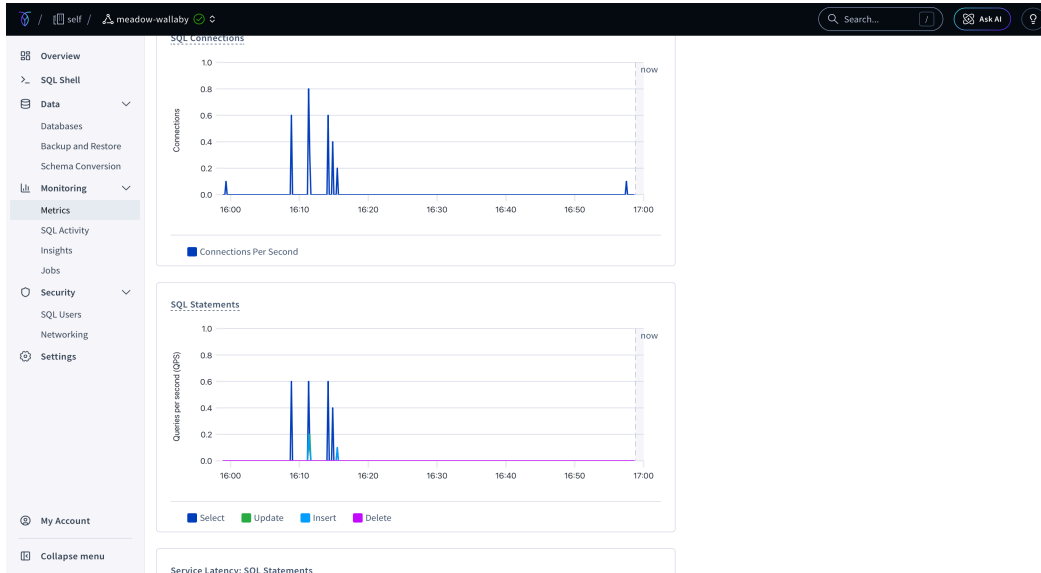
2. The SQL Shell Query (Proving the Agent's Memory)

- Where we went: We opened the SQL Console built into CockroachDB Cloud.
- What we captured: We ran the query `SELECT session_id, turn, role, content FROM conversations LIMIT 5;` and took a screenshot of the results.
- Why we did this: We did this to visualize the "Episodic Memory" of our agent. It serves as concrete proof that the chat history is actively being saved and persisted in CockroachDB.



3. The Cluster Overview / Metrics Dashboard

- Where we went: We navigated to the Overview and Metrics tabs for our cluster.
- What we captured: We took a screenshot of the SQL Statement graph and the general health dashboard showing active connections and queries currently being processed.
- Why we did this: We wanted our submission to look highly professional and prove that our Flask app is actively hitting the CockroachDB cloud cluster in real-time.



- Where we went: We went to Databases and clicked our database name to view the list of tables.
- What we captured: We took a screenshot of the list showing our core tables: courses, students, student_courses, conversations, and search_cache.
- Why we did this: We captured this because it perfectly aligns with the four points of "Agentic Memory" we outlined in our write-up. It proves to the judges that our schema was intentionally built to support RAG (Retrieval-Augmented Generation), identity state, and cognitive caching.

Additional Schemas/Metrics

- 
[self](#) / [meadow-mallaby](#) 

 SQL Shell

 Data

 Monitoring

 Security

 Settings

Databases

Databases

Database: defaultdb

public.recommendation_sessions

public.recommendation_sessions

Overview

Grants

Indexes

```

CREATE TABLE public.recommendation_sessions (
  id STRING NOT NULL,
  student_id STRING NULL,
  status STRING NULL DEFAULT 'in_progress'::STRING,
  created_at TIMESTAMPTZ NULL DEFAULT current_timestamp()::TIMESTAMPTZ,
  updated_at TIMESTAMPTZ NULL DEFAULT current_timestamp()::TIMESTAMPTZ,
  CONSTRAINT recommendation_sessions_pkey PRIMARY KEY (id ASC),
  CONSTRAINT recommendation_sessions_student_id_fkey FOREIGN KEY (student_id) REFERENCES public.students(id)
) WITH (schema_locked = true);
        
```

Size

954.5 KiB

Ranges

1

Replicas

3

% of Live data

97.4 %

Auto stats collections

Enabled

Stats last updated

Aug 17, 2026 at 1:14:53 UTC